IA Aplicada com LLMs

Aula 12 — DevOps

Objetivos

Ao final deste deck, espera-se que o aluno seja capaz de:

- **explicar** o problema que originou o DevOps — a separação entre desenvolvimento e operação, os ciclos longos de entrega e as falhas de implantação — e **definir** DevOps como cultura, práticas e medição, e não como cargo, equipe ou ferramenta;
- **formalizar** o ciclo vicioso da implantação rara: lote, probabilidade de falha e custo de diagnóstico como funções do intervalo entre implantações;
- **enunciar** os princípios pelo acrônimo CALMS e pelas três maneiras, e **distinguir** o que cada quadro descreve;
- **relacionar** cada prática ao princípio que ela realiza, com mecanismo e limite;
- **definir** SRE, SLI, SLO, orçamento de erro e *toil*, e **calcular** o orçamento de erro a partir do SLO.

Parte 1

O problema que originou o DevOps

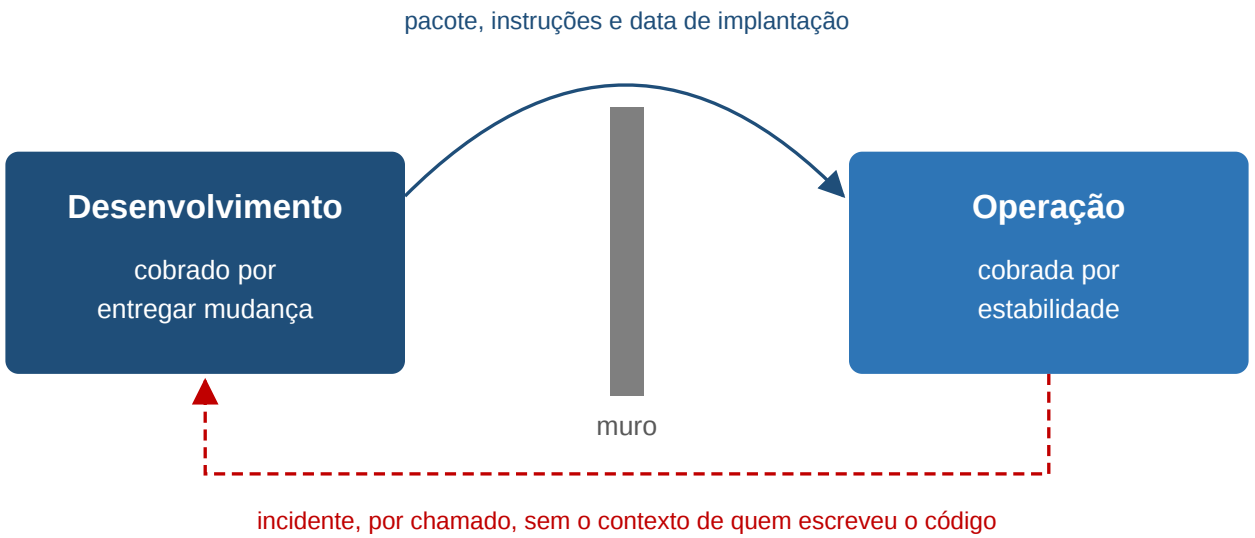
DevOps: cultura, práticas técnicas e medição

DevOps é o conjunto de práticas culturais, técnicas e de medição que reduz o tempo entre a decisão de mudar um sistema e a chegada da mudança à produção, mantendo ou aumentando a estabilidade em produção (KIM et al., 2016).

Componente	Conteúdo
cultura	responsabilidade compartilhada pelo sistema em produção entre quem o escreve e quem o opera
práticas técnicas	automação do caminho do repositório à produção, e do retorno de informação da produção ao repositório
medição	fluxo de entrega e comportamento em produção observados com números, e não com impressões

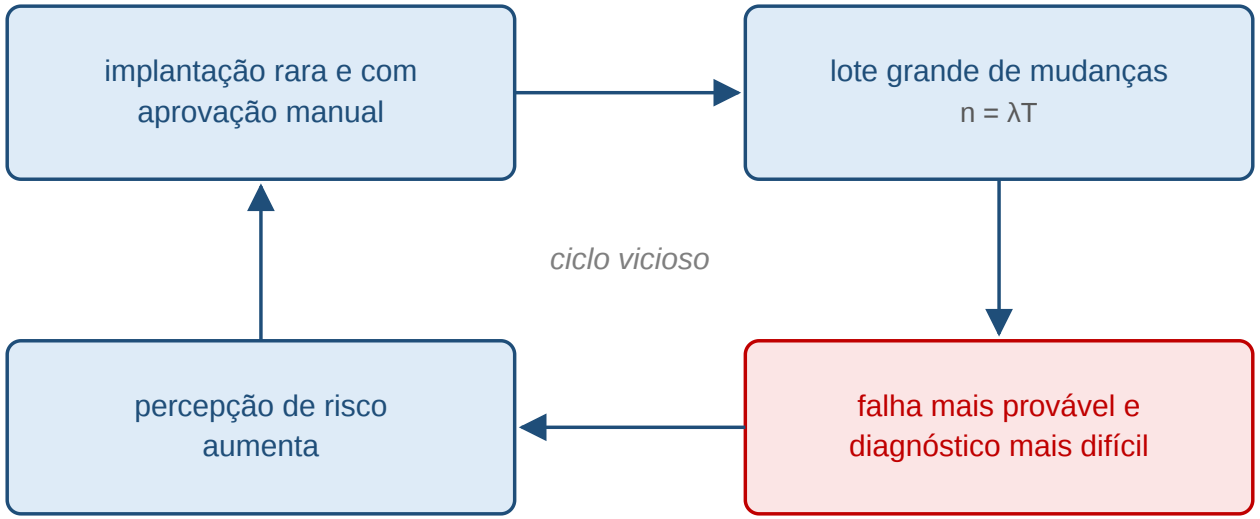
- **Não é cargo, equipe nem ferramenta.** Uma "equipe de DevOps" entre desenvolvimento e operação reproduz a separação, com um terceiro grupo no meio; um servidor de CI só automatiza uma prática, que só tem efeito se a organização em volta mudar.
- **Origem, 2009**: Allspaw e Hammond apresentam na Velocity as mais de dez implantações por dia da Flickr (ALLSPAW; HAMMOND, 2009); Patrick Debois organiza em Gante o primeiro DevOpsDays, que dá nome ao movimento.

O muro: incentivos opostos entre desenvolvimento e operação



- A mudança é o que o desenvolvimento produz e o que a operação tem razão para evitar: a maior parte dos incidentes em produção começa numa mudança (KIM et al., 2016).
- **Código "jogado por cima do muro"**: implantação manual em ambiente que o desenvolvimento não conhece, diagnóstico dividido entre dois grupos que se falam por chamado — e cada lado com incentivo para atribuir a falha ao outro.

O ciclo vicioso da implantação rara



- A raridade da implantação é **consequência e causa** do risco: cada falha leva a implantar menos e aprovar mais, e o ciclo volta com um lote maior.
- A resposta do DevOps inverte o ciclo: **automação** reduz o custo da implantação → implantação frequente → lote pequeno → risco menor por implantação.

O risco de cada implantação cresce com o intervalo

λ: mudanças por dia; T: dias entre implantações; p: probabilidade de uma mudança conter defeito que se manifesta em produção, com defeitos independentes.

$$n = \lambda T \qquad\qquad P_{\text{falha}}(T) = 1 - (1-p)^{\lambda T} \xrightarrow{T \to \infty} 1$$

- **Diagnóstico**: com n mudanças candidatas, a busca binária sobre o lote — reverter metade, testar, repetir — exige da ordem de $\lceil \log_2 n \rceil$ rodadas de implantação e verificação, supondo um único defeito e mudanças separáveis.
- **A independência é otimista**: mudanças acumuladas interagem, e uma interação não pertence a nenhuma mudança isolada. Com lote unitário, a falha aponta uma única mudança, e revertê-la é a correção imediata.
- **Velocidade e estabilidade não se trocam**: na pesquisa de Forsgren, Humble e Kim (2018), com milhares de profissionais em quatro anos, as organizações de alto desempenho implantam mais e mais rápido **e** falham menos e se recuperam mais depressa. As duas grandezas são efeitos da mesma causa — lote pequeno, automação e retorno rápido de informação.

Parte 2

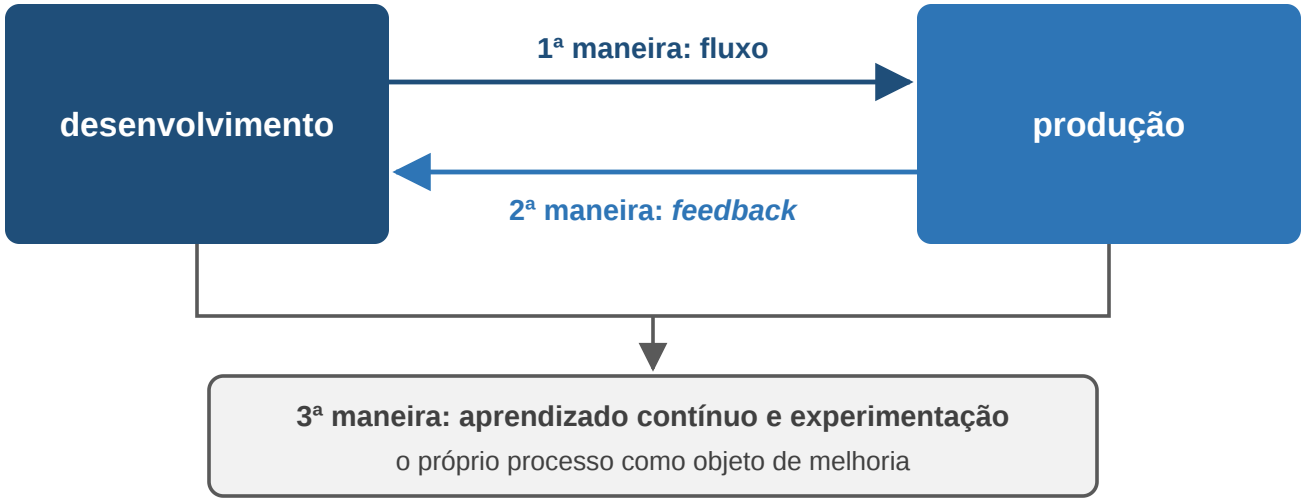
Princípios

CALMS: as dimensões que a organização precisa ter

Forma original **CAMS**, de John Willis e Damon Edwards (WILLIS, 2010); o **L**, de *lean*, foi acrescentado por Jez Humble.

	Dimensão	Definição	Prática que a realiza
C	Cultura (*Culture*)	responsabilidade compartilhada pelo sistema em produção	pós-incidente sem culpa; desenvolvimento no plantão
A	Automação (*Automation*)	todo passo repetível até a produção executado por máquina	integração contínua; *pipeline* de implantação; infraestrutura como código
L	*Lean*	lotes pequenos, limite de trabalho em andamento, sem espera nem retrabalho	implantações pequenas e frequentes; integração diária ao ramo principal
M	Medição (*Measurement*)	decisões tomadas sobre números coletados continuamente	monitoração e observabilidade; SLI e SLO
S	Compartilhamento (*Sharing*)	conhecimento, ferramentas e responsabilidade difundidos entre equipes	pós-incidente publicado; repositório e *pipeline* comuns

As três maneiras: direções ao longo do fluxo de valor



Fluxo de valor (*value stream*): a sequência de atividades entre a decisão de mudar o sistema e a mudança em produção (KIM et al., 2016).

1. **Fluxo** — lotes pequenos, limite de trabalho em andamento (WIP), gargalos visíveis, sem espera entre etapas.
2. ***Feedback*** — telemetria, testes automáticos e a regra de **parar a linha**: com o ramo principal quebrado, a integração seguinte não prossegue.
3. **Aprendizado** — análise de incidentes que muda o sistema, tempo reservado para melhorar o trabalho diário; depende das duas primeiras.

CALMS e as três maneiras descrevem aspectos diferentes

	CALMS	As três maneiras
Descreve	**o que** a organização tem	**em que direção** o trabalho e a informação circulam
Natureza	inventário de dimensões	princípios dinâmicos de onde as práticas derivam
Uso típico	diagnosticar o que falta	justificar por que uma prática existe
Exemplo	"não há medição contínua" — falta o M	"o defeito leva semanas para ser conhecido" — falha a 2ª maneira

Correspondências: Automação e *Lean* servem sobretudo ao **fluxo**; Medição serve ao ***feedback***; Cultura e Compartilhamento são a condição do **aprendizado**. Sem fluxo, um experimento leva meses para chegar à produção; sem *feedback*, o resultado dele não volta.

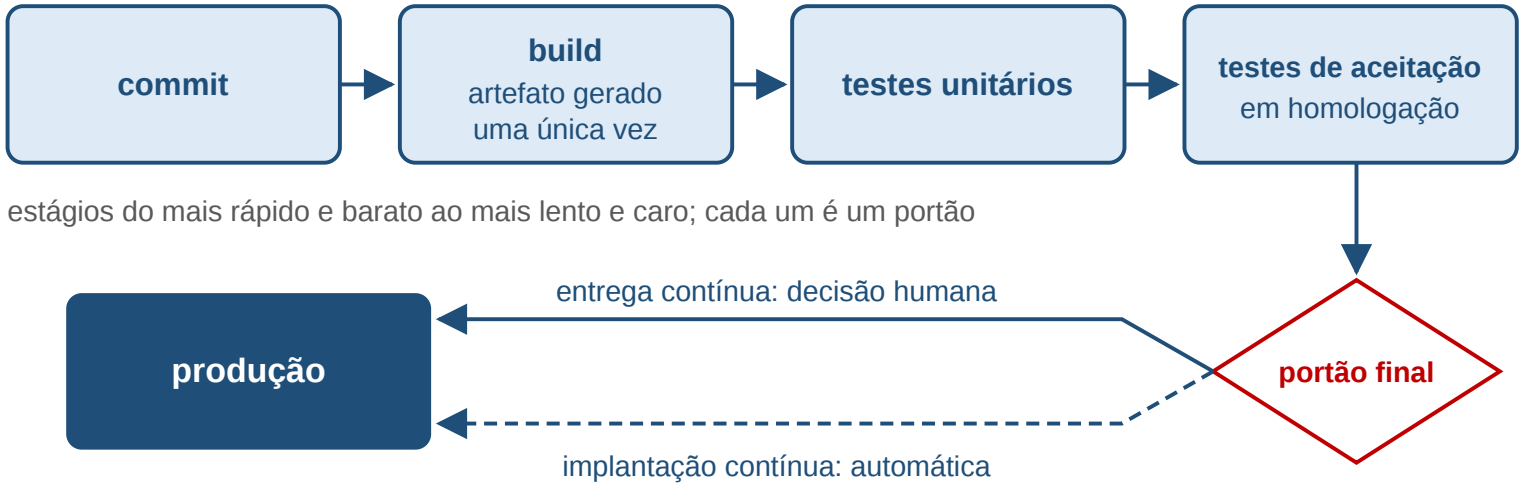
Parte 3

Práticas

Controle de versão de tudo e integração contínua

	Controle de versão de tudo	Integração contínua (CI)
Conceito	código, configuração, infraestrutura, migrações e *pipeline* versionados: o repositório é a **fonte única de verdade** (HUMBLE; FARLEY, 2010)	integrar ao ramo principal ao menos diariamente, com *build* e testes automáticos, mantendo-o sempre implantável (FOWLER, 2006)
Mecanismo	cada estado do sistema é uma revisão; "o que mudou entre ontem e hoje" é uma diferença entre duas revisões	ramo quebrado é a prioridade: corrigido ou revertido antes de qualquer outra integração; ramos curtos mantêm pequeno o lote de integração
Limite	o alterado à mão num servidor ou num painel não é reproduzível nem auditável; segredos ficam num cofre, e o repositório guarda só a referência	garante só o que os testes verificam; ramos longos com servidor de CI não são integração contínua
Princípio	pré-condição das demais; fluxo e compartilhamento	fluxo e *feedback*: o defeito é conhecido minutos depois da mudança

O *pipeline* de implantação: estágios como portões



- **Entrega contínua** (*continuous delivery*): toda mudança aprovada no *pipeline* **pode** ir à produção; a implantação é decisão de negócio, executada com um comando. **Implantação contínua** (*continuous deployment*): toda mudança aprovada **vai**, sem intervenção humana. A segunda pressupõe a primeira; a recíproca não vale (HUMBLE; FARLEY, 2010).

Artefato único, portões ordenados e reversão rápida

- **O artefato é construído uma única vez**, no primeiro estágio, e o mesmo binário é promovido de estágio a estágio; reconstruí-lo em cada ambiente abriria a possibilidade de testar uma coisa e implantar outra.
- **Os estágios vão do mais rápido e barato ao mais lento e caro**, para que a maior parte dos defeitos seja rejeitada cedo.
- **Limite**: o *pipeline* automatiza a decisão de promover e é tão confiável quanto os seus portões. A implantação contínua exige ainda reversão rápida ou liberação gradual, porque nenhum conjunto de testes antecipa todas as condições de produção.
- **Princípio**: fluxo — o caminho até a produção é automatizado e barato — e automação.

Infraestrutura como código e conteinerização

	Infraestrutura como código (IaC)	Conteinerização
Conceito	servidores, redes, bancos e permissões definidos em arquivos versionados (MORRIS, 2016)	**imagem**: artefato imutável com aplicação e dependências, identificado por *digest*; ***container***: processo isolado executado a partir dela
Mecanismo	o arquivo declara o estado desejado E_d; a ferramenta lê o real E_r e aplica só $\Delta = E_d \ominus E_r$. **Idempotente**: na segunda aplicação, $\Delta = \varnothing$. Servidor com defeito é recriado, não consertado (*cattle*, não *pet*)	a imagem é construída uma vez no *build* e o mesmo *digest* é promovido à produção — o artefato único levado ao ambiente inteiro; entre ambientes muda só a configuração injetada na partida
Limite	alteração manual cria **desvio de configuração** (*configuration drift*); dados persistentes não se recriam de uma definição	compartilha o núcleo do hospedeiro, sem o isolamento de uma máquina virtual; o estado gravado no *container* se perde com ele
Princípio	fluxo (ambientes sob demanda) e compartilhamento	fluxo e automação

Imagem, *container* e Compose: nota de Docker, da aula sobre o curso; a imagem publicada como serviço é tratada na aula de empacotamento.

Monitoração, observabilidade e pós-incidente sem culpa

Monitoração é a coleta contínua de dados quantitativos sobre o sistema em produção; **observabilidade** é a propriedade do sistema de permitir investigar o estado interno a partir da telemetria que emite. As duas são a 2ª maneira posta em prática — *feedback* e medição —, e o conteúdo delas é tratado na aula de observabilidade.

O **pós-incidente** (*postmortem*) é a análise escrita de um incidente resolvido; **sem culpa** (*blameless*), procura as condições sistêmicas que permitiram a falha, e não o indivíduo (ALLSPAW, 2012; BEYER et al., 2016).

- **Mecanismo informacional**: com punição, quem esteve envolvido deixa de relatar o que fez e por quê, e a organização perde a informação necessária para corrigir a condição. A pergunta passa de "quem errou" a "o que no sistema tornou esse erro possível" — um comando destrutivo sem confirmação, um painel sem o dado relevante —, e a ação corretiva muda o sistema.
- **Limite**: sem culpa não é sem responsabilidade — as ações corretivas têm responsável e prazo; e incidentes em sistemas complexos resultam em geral de vários fatores contribuintes, não de uma única causa raiz.
- **Princípio**: aprendizado contínuo, cultura e compartilhamento — o relatório publicado faz do incidente local aprendizado comum.

Cada prática realiza um princípio

Prática	Três maneiras	CALMS
Controle de versão de tudo	fluxo	Automação, Compartilhamento
Integração contínua	fluxo, *feedback*	Automação, *Lean*
Entrega e implantação contínuas	fluxo	Automação, *Lean*
Infraestrutura como código	fluxo	Automação, Compartilhamento
Conteinerização	fluxo	Automação
Monitoração e observabilidade	*feedback*	Medição
Pós-incidente sem culpa	aprendizado contínuo	Cultura, Compartilhamento

Parte 4

SRE como implementação

SRE: a operação tratada como engenharia de software

Site Reliability Engineering (SRE) é a forma como o Google opera os seus sistemas de produção: engenheiros que escrevem código para automatizar a operação e têm a confiabilidade do serviço como responsabilidade (BEYER et al., 2016). Em relação ao DevOps, é uma implementação concreta: os princípios aparecem como papéis, metas numéricas e regras de decisão.

Conceito	Definição	Exemplo
SLI (*service level indicator*)	medida quantitativa de um aspecto do serviço, em geral fração de eventos bons sobre eventos válidos	fração de requisições respondidas com sucesso em menos de 300 ms
SLO (*service level objective*)	valor-alvo de um SLI num intervalo de tempo	99,9% das requisições bem-sucedidas numa janela de 30 dias
orçamento de erro (*error budget*)	a quantidade de falha que o SLO admite	$B = 1 - \mathrm{SLO}$: 0,1% para um SLO de 99,9%

Orçamento de erro: a falha admitida pelo SLO

$$B = 1 - \mathrm{SLO} \qquad \text{eventos ruins admitidos} = B \cdot N \qquad \text{indisponibilidade admitida} = B \cdot \Delta t$$

Para um SLO de disponibilidade de 99,9% numa janela de 30 dias:

$$B = 1 - 0{,}999 = 0{,}001 \qquad B \cdot \Delta t = 0{,}001 \times 43\,200\ \text{min} = 43{,}2\ \text{min}$$

- **O SLO não é 100%**: nenhum usuário distingue 99,999% de 100% através das demais falhas do caminho até ele, e cada fração de disponibilidade acima do necessário custa velocidade de mudança.
- **Regra de decisão**: enquanto há orçamento, implanta-se; esgotado o orçamento, a prioridade passa a ser a estabilidade, e as implantações de funcionalidade são suspensas até a janela recuperá-lo (BEYER et al., 2016).
- **É a formalização do muro**: desenvolvimento e operação deixam de negociar caso a caso, porque compartilham a mesma meta numérica.

Toil e os limites do SRE

Toil é o trabalho operacional manual, repetitivo, automatizável, tático, sem valor duradouro e que cresce linearmente com o tamanho do serviço — reiniciar um processo à mão a cada alerta, aplicar a mesma alteração em cada servidor (BEYER et al., 2016).

- **Meta do Google**: *toil* abaixo de 50% do tempo de cada engenheiro de SRE; o restante vai para engenharia que reduz o *toil* futuro. É a 3ª maneira em forma de orçamento de tempo: sem ela, a operação cresce em proporção ao serviço e não sobra tempo para melhorar o processo.
- **Limite do indicador**: o SRE pressupõe SLIs que reflitam a experiência do usuário; um SLO cumprido sobre um indicador mal escolhido autoriza implantações enquanto os usuários percebem degradação.
- **Limite da regra**: o orçamento de erro só tem efeito se a organização suspender de fato as implantações quando ele se esgota.